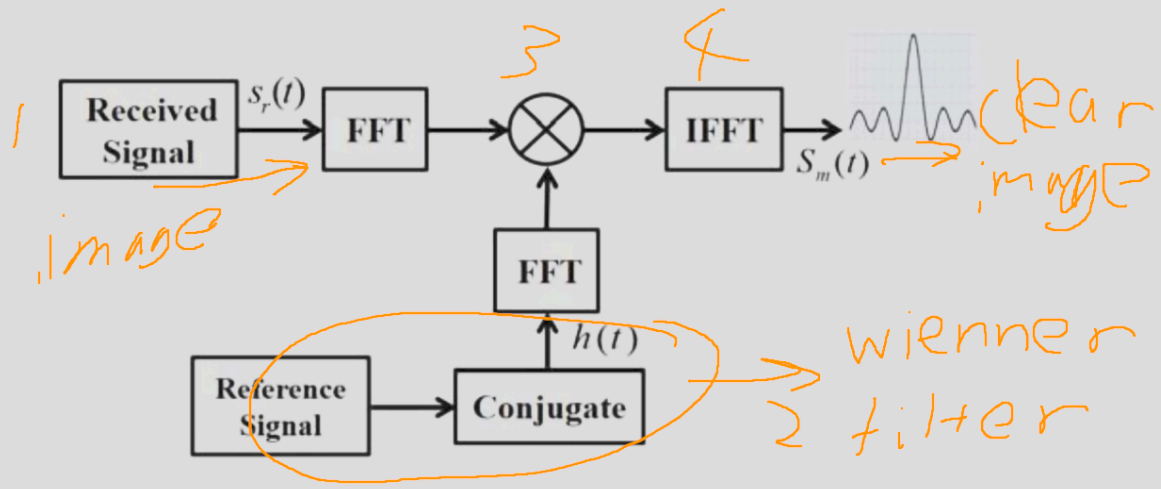


The following figure is the normal process of this convolution, where it will always be converted to frequency domain to save computation power by using the convolution property of Fourier transform.



Timeline:

1. Thursday Night: Finish individual module
2. Friday Night: Finish integration
3. Saturday: finish presentation
4. Sunday: the presentation day

# Frequency-Domain Image Deblurring Accelerator

## ASIC Specification Document

---

### 1. Overview

#### 1.1 Objective

The objective of this project is to design and implement an ASIC accelerator that performs image deblurring using frequency-domain filtering. The accelerator reconstructs blurred grayscale images by executing a Fast Fourier Transform (FFT), frequency-domain multiplication with a reconstruction filter, and an Inverse Fast Fourier Transform (IFFT).

This approach accelerates convolution-based image reconstruction by exploiting the convolution theorem, which states that convolution in the spatial domain corresponds to multiplication in the frequency domain.

The accelerator significantly improves performance compared to a CPU by leveraging pipelining, parallelism, concurrent processing, and optimized memory management.

---

## 1.2 Motivation

Image deblurring is a computationally intensive task due to:

- High computational complexity of convolution
- Large image sizes
- Poor cache locality on general-purpose CPUs
- Limited instruction-level parallelism

An ASIC accelerator can improve throughput and efficiency by implementing a dedicated hardware pipeline optimized for FFT-based filtering.

---

## 1.3 Target Application

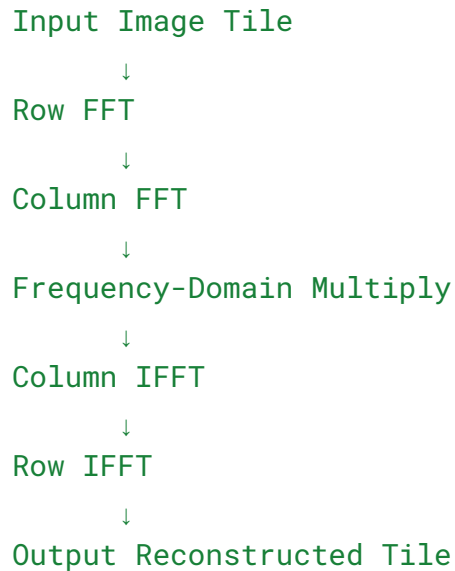
This accelerator targets applications such as:

- Image reconstruction
  - Satellite and aerial image processing
  - Computer vision preprocessing
  - Scientific imaging
  - Real-time image enhancement systems
- 

# 2. System Architecture

## 2.1 Top-Level Pipeline

The accelerator implements the following processing pipeline:



Each stage operates in a pipelined and streaming manner to maximize throughput.

---

## 2.2 Functional Modules

### 1. FFT module (George)

Input: 2D tile of spatial-domain image samples (grayscale).

Output: 2D tile of frequency-domain complex values  $Y(u,v)$  in a defined bin order (e.g., row-major).

Notes: Internally treats input as complex with  $\text{imag}=0$ ; outputs complex frequency bins aligned to the multiplier's expected order.

### 2. FilterGen module (Jesscia)

Input: Filter mode/config (e.g., Wiener/inverse/CLS), optional blur descriptor or precomputed blur spectrum  $H(u,v)$ , optional image/tile statistics (e.g., noise/blur estimate), and tile size  $N \times N$ .

Output: 2D tile (stream) of complex filter coefficients  $G(u,v)$  in the same bin order as the FFT output; plus valid/ready handshake.

Notes: Implemented either as (A) filter-bank selection from ROM/SRAM or (B) on-chip coefficient generation; output must be time-aligned or order-aligned with the FFT bins.

### 3. Multiplier module (Belle - but up to switch if you have a strong preference @chris)

Input: Frequency-domain complex tile  $Y(u,v)$  from FFT, and complex filter tile  $G(u,v)$  from FilterGen (same dimensions and bin order).

Output: Filtered frequency-domain complex tile  $Y_f(u,v) = Y(u,v) * G(u,v)$ .

Notes: Element-wise complex multiply; supports parallel lanes; assumes  $Y$  and  $G$  are synchronized in bin order.

### 4. IFFT module

Input: Filtered frequency-domain complex tile  $Y_f(u,v)$ .

Output: Reconstructed spatial-domain tile  $\hat{x}(x,y)$  as complex values; final image uses the real component (image discarded/ignored) and is converted/clipped to output pixel format.

Notes: Output is the deblurred image tile in spatial domain, same dimensions as input tile.

---

## 3. Functional Specification

---

### 3.1 Input Interface

#### Function

Receives blurred grayscale image tiles and stores them into on-chip memory.

## Input Format

- Image format: grayscale
- Pixel precision: 8-bit unsigned integer
- Internal representation: fixed-point complex format

## Tile Size

Default tile size:

64 × 64 pixels

Tile-based processing allows efficient use of on-chip memory and enables streaming operation.

---

## 3.2 FFT Processor

### Function

Transforms image data from spatial domain to frequency domain using a 2D FFT.

The 2D FFT is implemented as:

- Row FFTs
- Column FFTs

### Input

Complex pixel values from tile buffer.

### Output

Frequency-domain representation of the image tile.

### Implementation Features

- Radix-2 FFT architecture
  - Fixed-point arithmetic
  - Pipeline stages for each FFT stage
  - Multiple butterfly units for parallel computation
- 

### 3.3 Frequency-Domain Multiply Unit

#### Function

Applies the reconstruction filter by performing element-wise complex multiplication:

$$Y(u,v) \cdot G(u,v)$$

Where:

- $Y(u,v)$  is the FFT of the blurred image
- $G(u,v)$  is the precomputed reconstruction filter

#### Implementation Features

- Parallel complex multiplier lanes
- Fully pipelined multipliers
- Filter coefficients stored in on-chip ROM or SRAM

#### Complex Multiply Operation

$$(a+jb)(c+jd) = (ac-bd) + j(ad+bc)$$

---

### 3.4 IFFT Processor

#### Function

Transforms frequency-domain data back to spatial domain using a 2D inverse FFT.

## **Output**

Reconstructed image tile with blur removed or reduced.

## **Implementation**

Same architecture as FFT processor, with inverse transform configuration.

---

# **3.5 Output Interface**

## **Function**

Writes reconstructed image tiles to output memory.

## **Output Format**

- Grayscale pixels
  - Fixed-point converted to 8-bit output
- 

# **4. Architectural Optimizations**

The accelerator improves performance using four primary architectural techniques.

---

## **4.1 Pipelining**

Each processing stage is implemented as a pipeline.

Pipeline stages include:

- FFT stages

- Multiply stages
- IFFT stages

Pipeline operation enables continuous processing of image tiles without idle cycles.

---

## 4.2 Parallelism

Parallelism is implemented at multiple levels:

### Butterfly-Level Parallelism

Multiple FFT butterfly units operate simultaneously.

### Multiply-Level Parallelism

Multiple frequency bins are processed per clock cycle.

This increases throughput significantly.

---

## 4.3 Concurrent Processing

The accelerator uses stage-level concurrency.

While one tile is processed in the FFT stage, another tile can be processed in the multiply stage, and another tile in the IFFT stage.

Example:

```
FFT      → Tile 3  
Multiply → Tile 2  
IFFT     → Tile 1
```

This enables continuous data flow and maximizes hardware utilization.

---

## 4.4 Memory Management Optimization

Efficient memory architecture is used to prevent memory bottlenecks.

Techniques include:

- Double buffering (ping-pong buffers)
- Banked SRAM architecture
- Streaming memory access
- Dedicated transpose buffers for column FFT

These techniques ensure continuous data supply to processing units.

---

# 5. Memory Architecture

## 5.1 On-Chip Memory Components

The accelerator uses the following memory structures:

- Input tile buffer
  - FFT intermediate buffer
  - Multiply buffer
  - IFFT intermediate buffer
  - Output buffer
  - Filter coefficient memory
- 

## 5.2 Double Buffering

Two buffers are used per stage:

- Buffer A: processing
- Buffer B: loading next tile

Buffers swap roles after completion.

This prevents pipeline stalls.

---

## 6. Control Unit

The control unit manages:

- Pipeline scheduling
- Memory access coordination
- Module synchronization
- Tile sequencing

The control unit ensures correct data flow through the pipeline.

---

## 7. Data Precision

The accelerator uses fixed-point arithmetic for efficiency.

Typical configuration:

- Input: 8-bit integer
- Internal: 16-bit fixed-point
- Intermediate: 16–24 bit precision

- Output: 8-bit integer

Fixed-point arithmetic reduces hardware cost while maintaining sufficient accuracy.

---

## 8. Performance Goals

Target performance characteristics:

- Tile size:  $64 \times 64$
  - Fully pipelined operation
  - Continuous throughput after pipeline fill
  - Significantly higher throughput than CPU implementation
- 

## 9. Advantages over CPU Implementation

The ASIC accelerator provides performance improvements through:

- Dedicated hardware datapath
- Streaming pipeline architecture
- Parallel processing units
- Optimized memory architecture
- Elimination of instruction overhead

CPU implementations suffer from:

- Sequential execution

- Cache inefficiencies
  - Limited parallelism
  - Higher latency
- 

## 10. Verification Strategy

Verification includes:

- Functional simulation using golden reference model
  - Comparison with software FFT implementation
  - Image reconstruction accuracy validation
  - Pipeline correctness verification
- 

## 11. Future Extensions

Possible future enhancements include:

- Support for larger tile sizes
  - Configurable reconstruction filters
  - Color image support
  - Higher parallelism configurations
  - Integration into real-time image processing systems
-

## 12. Summary

This accelerator implements an optimized hardware architecture for image deblurring using FFT-based frequency-domain filtering.

Key architectural strengths include:

- Fully pipelined design
- Parallel FFT and multiply units
- Concurrent processing across pipeline stages
- Optimized memory architecture

This design demonstrates how specialized hardware accelerators can significantly outperform general-purpose CPUs for signal and image processing workloads.